Exercise: Indexes on Real Data

Fifty thousand made-up pets show how an index works. They do not show why anyone needs one. In Chapter 9 you loaded the IMDb title and rating files into SQLite, about 12.8 million titles and 1.7 million ratings, and measured what indexes did to real queries. In this exercise you load the same files into Atlas and take the same kind of measurements in MongoDB. The timings are yours to collect. This chapter does not give you numbers to match.

Get the IMDb files

You need two compressed files: `title.basics.tsv.gz` and `title.ratings.tsv.gz`. You have two ways to get them.

Reuse your old files. If you did Chapter 9 and still have the `data` folder, copy both files into a `data` folder next to this chapter's code. Keep them compressed. The loader reads them directly.

Download them again. From the Chapter 14 `code` folder:

```
mkdir -p data
curl -fL -o data/title.basics.tsv.gz https://datasets.imdbws.com/title.basics.tsv.gz
curl -fL -o data/title.ratings.tsv.gz https://datasets.imdbws.com/title.ratings.tsv.gz
```

The source and the terms of use are at https://developer.imdb.com/non-commercial-datasets/. The files are for personal, non-commercial use. Do not redistribute them or put them in a public repository. As in Chapter 9, add `**/data/` to your root `.gitignore` before you download, and check it with `git check-ignore -v data/title.basics.tsv.gz`.

Check your storage first

This is a lot more data than the pets exercise. The two collections together hold roughly 14 million documents, and every index you build adds more. The Atlas Free tier has a small storage limit, and this data will probably not fit. Look at your cluster's storage limit and the size of the collections before you start. If the full data will not fit, load a subset. Both the loader and the experiments work on any number of documents, and the lessons are the same on a smaller collection, although the gaps are smaller too. Load into a database named `imdb`, separate from your `pets` database, so you can drop it when you are done.

Load the data

The supplied `import-imdb-mongo.py` reads the two files one row at a time and inserts documents into Atlas in batches of 10,000. The IMDb identifier, such as `tt0000001`, becomes the document's `_id`, so MongoDB's automatic `_id` index does the job the primary key did in SQLite. The loader creates no other indexes. As in Chapter 9, we load first and index afterward, because building an index on a loaded collection is cheaper than maintaining it during the load.

Try This: With `MONGODB_URI` set, run:

```
python3 import-imdb-mongo.py --limit 100000
```

This loads at most 100,000 documents per collection. Remove `--limit 100000` only when the cluster has enough storage for the full files. A subset may omit The Godfather; zero matches still provide a useful comparison of documents examined.

It prints progress and the time for each file. It refuses to overwrite existing collections unless you add `--replace`. Write down how long each file took. Then confirm the counts in `mongosh`:

```
use imdb
db.titles.countDocuments({})
db.ratings.countDocuments({})
```

Run the experiments

The supplied `explore-imdb-indexes.py` runs three experiments. For each one it prints the query, the plan stage MongoDB chose, the documents examined and returned, and the best of three timings. It then builds an index, reports how long the build took, and measures the same query again.

Experiment	Query	Index it builds
One title	`primary_title` equals “The Godfather”	`primary_title`
Two conditions	`title_type` is movie and `start_year` is 1994	`title_type`, then `start_year`
Top ten by votes	ratings sorted by `num_votes`, descending, limit 10	`num_votes`, descending

Try This:

```
python3 explore-imdb-indexes.py
```

The script creates only indexes whose names start with `idx_` and drops them when it starts, so you can run it again. It does not change your data. Your timings will depend on your cluster tier, your network, and whether Atlas already has the data in memory. Run the script more than once and note the difference between the first run and the later ones.

Record what you measured

Copy the table below into your notes and fill it in from the script’s output. Use the server’s `executionTimeMillis` if you also run the queries by hand in `mongosh` with `explain("executionStats")`, because it leaves out network delay.

Experiment	Plan before	Examined before	Time before	Index build time	Plan after	Examined after	Time after
One title							
Two conditions							
Top ten by votes							

Also record how long each file took to load, and the size of each index. In `mongosh`:

```
db.titles.stats().indexSizes
db.ratings.stats().indexSizes
```

Go further

These are the same questions Chapter 9 asked of SQLite. Answer each by running the query in `mongosh` with `explain("executionStats")`, building or dropping indexes as needed.

1. Create an index on `start_year` alone and run the two-condition query. Then replace it with a compound index on `title_type` and `start_year`. Compare the documents examined. Which order of the two fields would you choose, and what does the plan show if you reverse it?
2. Query on `start_year` alone with the compound index in place. Does MongoDB use the index? Explain what you see in terms of the leading field.
3. Run `{primary_title: /^The Godfather/}` and then `{primary_title: /Godfather/}` with an index on `primary_title`. Which one can use the index to narrow the search, and why? Compare what you see with the LIKE results from Chapter 9.
4. Create an index on `num_votes` in `ratings` and run the top-ten query with and without `limit(10)`. What does the limit change about the work done?
5. Compare your index sizes and build times. Which index cost the most storage? Which cost the most time? What does that tell you about indexing a long text field such as `primary_title`?
6. Insert 100,000 copies of an existing document's fields, with new `_id` values, once with no extra indexes and once with three. Compare the times. Chapter 9 measured the same effect in SQLite.
7. Compare your MongoDB results with your Chapter 9 results. Where did the same index help in the same way, and where did the two systems behave differently?

Clean up

When you finish, drop the experiment database so it stops using your storage:

```
use imdb
db.dropDatabase()
```

The IMDb files stay on your computer, so you can reload them whenever you need to.